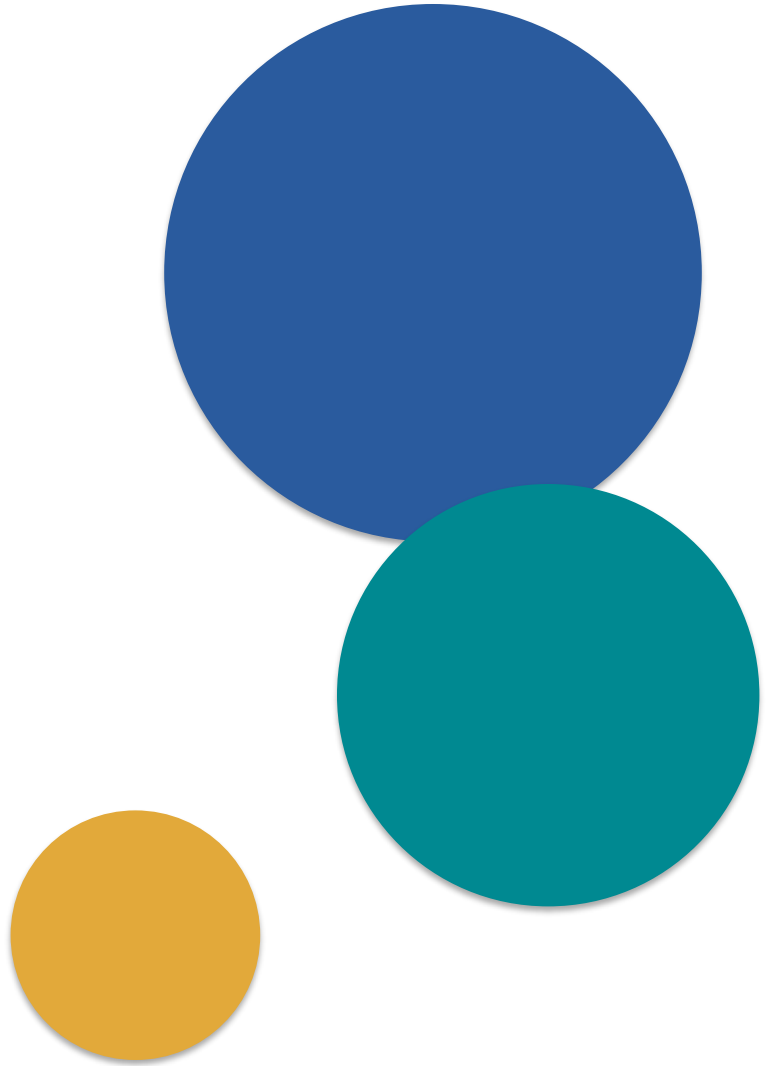
Computer Hardware, Software & Architecture

Engineering Information Systems and Digital Skills

Lecture 2 | First-year Engineering Students

60 minutes teaching + 30 minutes interaction & practice



By the end of this lecture, you should be able to...

Learning outcomes

- Identify the main functional components of a computer system.
- Explain the roles of the CPU, ALU, Control Unit, registers, cache, RAM and storage.
- Describe the instruction cycle and the purpose of buses.
- Explain the memory hierarchy and basic factors affecting performance.
- Distinguish system software from application software.
- Recognize the roles of operating systems, programming languages and translation tools.
- Connect hardware and software to common engineering applications.

Why do engineers need to understand hardware & software?

From physical components to the software that makes engineering systems work

Hardware provides resources

- CPU and memory process information.
- Sensors and input devices collect data.
- Displays and actuators present results or act on the world.
- Storage keeps files and programs.

Software provides instructions

- Operating systems manage resources.
- Applications solve specific tasks.
- Programs analyze data, model systems and control devices.
- Engineers need both to select, use and troubleshoot systems.

What is a computer system?

A computer system is more than a CPU.

Key points

- Accepts input: data or commands enter the system.
- Processes data: instructions transform input into useful results.
- Produces output: results go to a user or another system.
- Stores information: data and programs are retained for later use.
- Uses control mechanisms: hardware and software coordinate activities.

A computer system: the main functional groups

Control coordinates and manages the system

1 INPUT

Receives data and commands

Keyboard • mouse • scanner

Sensors • camera

2 PROCESSING

Transforms data

CPU • ALU • Control Unit

Registers • cache

3 MEMORY & STORAGE

Holds active data and saved files

RAM • cache • SSD • HDD

4 OUTPUT

Presents results

Monitor • printer • display

Speakers • actuators

CONTROL CPU and system software coordinate system activities.

Computer architecture and computer organization

Architecture describes what the system supports; organization describes how it is built.

Computer architecture

The features visible to a programmer or system designer.

Examples:

- Instruction set
- Data types and supported operations
- Addressing methods

Computer organization

How the hardware is arranged to implement the architecture.

Examples:

- CPU components and internal connections
- Cache design
- Buses and memory technology

The main components inside a computer

CPU

Executes instructions and coordinates processing.

MAIN MEMORY

Holds programs and data currently in use.

STORAGE

Keeps files and programs when power is off.

INPUT / OUTPUT

Connects the computer to users, networks and physical devices.

Inside the CPU: the processing core

The CPU depends on memory, storage, input/output devices and software.

What the CPU does

- Executes instructions.
- Performs processing operations.
- Coordinates basic activities needed to run programs.

Closely connected components

- ALU: arithmetic and logic.
- Control Unit: directs instruction execution.
- Registers: very small, fast storage.
- Cache: keeps frequently used information close to the CPU.

Inside the CPU: Arithmetic Logic Unit (ALU)

The ALU performs arithmetic and logical operations required by instructions.

Arithmetic operations

Addition, subtraction, multiplication and division.

Example: calculate the average temperature from several sensor readings.

Logical operations

Comparisons such as greater than, less than or equal to.

Example: check whether a measured temperature is above a safety limit.

Inside the CPU: Control Unit (CU)

Fetch → interpret/decode → control → execute

What it does

- Directs and coordinates CPU activities.
- Fetches and interprets instructions.
- Controls the sequence of operations.

How it coordinates

- Sends control signals to the ALU.
- Coordinates registers and memory.
- Helps coordinate other computer components.

It manages the sequence; it does not perform the main arithmetic calculations.

Registers and cache: keeping information close

Both help the CPU access useful information quickly.

Registers

- Very small storage locations inside the CPU.
- Hold data, instructions, addresses or intermediate results.
- Give the CPU immediate access to information it is using.

Cache

- High-speed memory close to the CPU.
- Stores frequently used data and instructions.
- Reduces CPU waiting time.
- Smaller than RAM, but typically faster.

The instruction cycle

The cycle repeats while a program runs.

1 FETCH

The CPU obtains the next instruction from memory.

2 DECODE

The instruction is interpreted to determine what is required.

3 EXECUTE

The CPU performs the required operation.

4 STORE / UPDATE

The result is stored or the program state is updated.

Buses: moving data and signals

A bus is a communication path inside a computer.

DATA BUS

Carries data between components.

ADDRESS BUS

Identifies the location being accessed.

CONTROL BUS

Carries signals that coordinate operations.

WHY IT MATTERS

Components must exchange data and signals to work as one system.

Memory vs. storage

Think: active working space versus long-term retention.

Memory (RAM and cache)

- Supports programs while they are running.
- Fast access to active data.
- RAM is usually volatile: its contents are lost when power is off.
- Cache is smaller and closer to the CPU.

Storage (SSD and HDD)

- Keeps files and programs long term.
- Retains information when power is off.
- Usually has more capacity than RAM.
- SSDs have no moving parts and are generally faster than HDDs.

Memory hierarchy: speed, size and cost

In general, moving closer to the CPU means faster access and smaller capacity.

REGISTERS

Smallest and fastest; inside the CPU.

CACHE

Very fast; keeps frequently used data near the CPU.

RAM

Larger working area for active programs and data.

SSD / HDD

Long-term storage; usually larger but slower than RAM.

Peripherals: input, output and communication

Peripherals connect a computer to users and the wider world.

INPUT

Keyboard, mouse, scanner, camera and sensors.

OUTPUT

Monitor, printer, speakers, displays and actuators.

COMMUNICATION

Network interface, Wi-Fi and Bluetooth.

ENGINEERING

Data-acquisition devices, sensors, controllers and actuators.

Computer systems: from PCs to specialized systems

Specialized and embedded computers are designed for a specific function.

PERSONAL COMPUTERS

Desktops and laptops for documents, CAD, programming and analysis.

SERVERS

Provide files, databases, applications and other services to many users.

MAINFRAMES

Process very large volumes of critical organizational transactions.

SUPERCOMPUTERS

Support demanding scientific calculations, modelling and simulations.

Specialized and embedded computers

An embedded computer is part of a larger product or system.

Examples

- Microcontrollers in small devices.
- Automotive control systems.
- Industrial controllers and PLC-based systems.
- Smart sensors and connected equipment.

Typical characteristics

- Designed for a defined task.
- May have limited memory and power.
- Often interacts with sensors and actuators.
- May need predictable or real-time responses.

What affects computer performance?

Performance depends on the whole system, not one specification alone.

CPU

Processing design and clock rate affect how quickly operations can be handled.

CORES

Multiple cores can support work in parallel when software allows it.

CACHE & RAM

Fast access to data can reduce waiting during processing.

STORAGE & SOFTWARE

Storage speed and efficient software affect loading and overall responsiveness.

CPU cores and multicore processing

Example: a workstation may run a simulation while managing other tasks.

A CPU core

A processing unit capable of executing instructions.

A processor may contain one or more cores.

Why multiple cores help

- Different tasks may run at the same time.
- Software designed for parallel work can benefit.
- More cores do not automatically make every program faster.

RISC and CISC: two instruction-set approaches

The instruction set is part of computer architecture.

RISC

Reduced Instruction Set Computer.

Often uses a relatively small set of simple instructions designed for efficient execution.

CISC

Complex Instruction Set Computer.

Includes instructions that can perform more complex operations.

Both approaches are used in real processors; performance depends on the complete design.

What is software?

Software is a set of instructions that tells hardware what to do.

System software

Manages computer resources and provides a platform for applications.

Examples: operating systems, utilities and device drivers.

Application software

Helps users complete specific tasks.

Examples: CAD, spreadsheets, browsers, MATLAB and engineering simulation tools.

Common application software

GENERAL PURPOSE

Word processing, spreadsheets, presentations, browsers and email.

ENGINEERING DESIGN

CAD and computer-aided manufacturing tools.

ANALYSIS & SIMULATION

MATLAB, numerical tools, modelling and simulation software.

DATA & DEVELOPMENT

Data analysis tools, databases and programming environments.

Operating system: the manager of the computer

Applications normally rely on the operating system to access resources.

Main functions

- Provides a user interface.
- Manages processes and tasks.
- Allocates memory.
- Organizes files and storage.
- Manages devices.
- Supports security and access control.

Examples

Windows • Linux • macOS • Android

The operating system manages hardware resources and provides services for applications and users.

Hardware + software = a working computer system

Example: an engineering simulation needs both capable hardware and suitable software.

Hardware provides resources

CPU → executes calculations

RAM → holds active data

Storage → keeps files and programs

Input devices → collect data

Output devices → present results

Software directs the resources

Operating system → manages resources

Application → performs a task

Program → supplies instructions

Drivers → help the OS communicate with devices

Programming languages: why engineers need them

Examples: data analysis, embedded systems, optimization and numerical calculations.

What is a programming language?

A formal way to give instructions to a computer.

Programs turn a problem into a sequence of instructions the computer can execute.

Why engineers use programming

- Automate repetitive calculations.
- Process and analyze engineering data.
- Build models and simulations.
- Control and automate devices.

Programming languages: from machine to high-level

MACHINE LANGUAGE

Instructions represented at the machine level; difficult for humans to write.

ASSEMBLY LANGUAGE

Uses symbolic instructions and is closely related to hardware.

HIGH-LEVEL LANGUAGES

Closer to human language and easier to use for complex applications.

MAIN IDEA

Programs written by people must be translated or interpreted so hardware can execute them.

How programs run: translation tools

COMPILER

Translates a program into a form that can be executed.

INTERPRETER

Reads and executes program instructions during execution.

ASSEMBLER

Translates assembly-language instructions into machine code.

SIMPLE VIEW

Human-readable program → translation or execution tool → instructions executed by the computer.

Python, C/C++ and MATLAB

PYTHON

A good introductory choice for data analysis, automation and AI.

C / C++

Efficient and widely used for embedded systems and hardware-oriented applications.

MATLAB

Useful for numerical computing, matrices, modelling, simulation and data analysis.

IN THIS COURSE

The goal is to recognize each tool's purpose; detailed programming is studied separately.

Engineering applications: design and automation

Hardware and software work together in engineering environments.

CAD / design

Hardware: workstation, graphics processor and display.

Software: CAD package for 2D and 3D design.

Industrial automation

Hardware: PLC, sensors, actuators and communication devices.

Software: control program and human-machine interface (HMI).

Engineering applications: data and simulation

Choose resources based on the task, data size and performance needs.

Data acquisition

Hardware: sensor, data-acquisition interface and computer.

Software: acquisition and analysis tools.

Flow: physical measurement → digital data → analysis.

Engineering simulation

Hardware: CPU, RAM and storage.

Software: MATLAB, numerical tools or simulation package.

The computer processes a model to estimate system behaviour.

Worked example: a temperature-control system

Hardware collects and acts on signals; software defines the decision logic.

1 MEASURE

A temperature sensor measures the process.

2 INPUT

An interface sends the measurement to a controller.

3 PROCESS

The program compares the value with a target or limit.

4 OUTPUT / ACTION

A display shows the result; a controller may activate an actuator.

30-minute interaction & practice

Discuss choices and explain your reasoning.

Activity A — choose the tool (15 min)

In small groups, choose a suitable programming tool for:

1. A robot controller.
2. Engineering data analysis.
3. Numerical simulation.

Give one reason for each choice.

Activity B — follow the system (15 min)

For a temperature-control system, identify:

- Input device
- Processing component
- Memory or storage needed
- Software involved
- Output or action

Explain how the parts work together.

Quick practice: choose the right resource

Answer individually, then compare with a partner.

Key points

- You need to keep a large CAD project when the computer is turned off. What resource?
- A running simulation needs temporary working space. What resource?
- A device measures temperature and sends the value to a computer. What hardware role?
- What software manages files, memory, devices and applications?
- Which is an application: Windows, RAM, MATLAB or CPU?

Lecture 2 Review: Multiple Choice Questions — 1.1

Choose the best answer. Do not check the answer key until the end.

Question 1

Which component performs arithmetic and logical operations?

A. Control Unit

B. Cache

C. Storage

D. ALU

Question 2

What is the main role of the Control Unit (CU)?

A. Store files permanently

B. Display information

C. Coordinate CPU activities

D. Perform arithmetic calculations

Lecture 2 Review: Multiple Choice Questions — 1.2

Choose the best answer. Do not check the answer key until the end.

Question 3

Which memory is located close to the CPU and stores frequently used data?

A. SSD

B. USB drive

C. Optical disk

D. Cache

Question 4

Which of the following is an example of system software?

A. Presentation software

B. Operating system

C. CAD software

D. MATLAB

Lecture 2 Review: Multiple Choice Questions — 2.1

Choose the best answer. Do not check the answer key until the end.

Question 5

What is the main purpose of an operating system?

A. Manage hardware resources and provide services

B. Design mechanical parts

C. Perform only mathematical calculations

D. Replace the CPU

Question 6

Which language is especially suitable for embedded systems and hardware-oriented applications?

A. MATLAB

B. C/C++

C. Spreadsheet formulas

D. HTML

Lecture 2 Review: Multiple Choice Questions — 2.2

Choose the best answer. Do not check the answer key until the end.

Question 7

Which language is a good introductory choice for data analysis and automation?

A. Assembly

B. HTML

C. Python

D. Machine language

Question 8

Which tool is widely used for numerical computing, modelling and simulation?

A. MATLAB

B. Operating system

C. Assembly

D. BIOS

Lecture 2 Review: Multiple Choice Questions — 3.1

Choose the best answer. Do not check the answer key until the end.

Question 9

In a data-acquisition system, what usually collects the physical measurement?

A. Sensor

B. Operating system

C. Cache

D. Compiler

Question 10

In industrial automation, which device commonly executes a control program?

A. Keyboard

B. Printer

C. PLC

D. Monitor

Lecture 2 Review: Multiple Choice Questions — 3.2

Choose the best answer. Do not check the answer key until the end.

Question 11

Which statement best describes a compiler?

A. It measures temperature

B. It controls the display

C. It stores data permanently

D. It translates a program into a form that can be executed

Question 12

Which engineering application uses CAD software?

A. CPU instruction fetching

B. 2D and 3D engineering design

C. Operating-system management

D. Network cabling only

Lecture 2 Review: Answer Key

Check your answers after completing all 12 questions.

Questions 1–4

1. D — ALU
2. C — Coordinate CPU activities
3. D — Cache
4. B — Operating system

Questions 5–8

5. A — Manage hardware resources
6. B — C/C++
7. C — Python
8. A — MATLAB

Questions 9–12

9. A — Sensor
10. C — PLC
11. D — Compiler
12. B — 2D and 3D design

Lecture summary

Remember the relationships, not only the component names.

Key points

- A computer system combines input, processing, memory/storage, output and control.
- The CPU uses the ALU, Control Unit, registers and cache to execute instructions.
- The instruction cycle includes fetching, decoding and executing instructions.
- Memory supports active work; storage keeps files long term.
- Operating systems manage resources; applications perform tasks.
- Programming tools support analysis, simulation, automation and embedded systems.
- Engineering performance depends on the complete hardware–software system.

Exit ticket & next lecture

One-minute individual reflection

Before you leave

Write one sentence for each:

1. What is one difference between RAM and storage?
2. How are hardware and software different?
3. Which programming tool would you choose for one engineering task, and why?

Next lecture: Data Resource Management

How data is organized, stored, managed, protected and transformed into useful information.

Bring one example of engineering data and consider how it should be stored.